

METODOS NUMERICOS

TEMA 4

Felipe Carrasco Escalante



Introducción

Los **métodos numéricos** son herramientas fundamentales en la ingeniería para aproximar soluciones de problemas matemáticos que, en muchos casos, son difíciles o imposibles de resolver de manera analítica. En el presente trabajo, se abordan dos áreas clave: la **integración numérica** y la **diferenciación numérica**.

Para la integración, se implementan los métodos de **Trapecio**, **Simpson 1/3** y **Simpson 3/8**, los cuales permiten calcular el área bajo una curva mediante la división del intervalo en segmentos geométricos. Por otro lado, en la diferenciación numérica, se utilizan las fórmulas de **tres y cinco puntos**, que aprovechan los valores de la función en puntos cercanos para estimar la razón de cambio o derivada con altos niveles de precisión. Cada algoritmo se presenta con su respectivo código en **Python**, su validación de ejecución y una explicación detallada de los cálculos realizados.

1. El Método del Trapecio

Ejercicio 1:

Código del trapecio en Python

```
def f(x):  
    return x**2  
  
def metodo_trapecio(a, b):  
    return (b - a) * (f(a) + f(b)) / 2  
  
resultado = metodo_trapecio(0, 3)  
print(f"Resultado aproximado de la integral: {resultado:.5g}")
```

Ejecución:

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/  
felip/Downloads/trapecio.py  
Resultado aproximado de la integral: 13.5
```

Explicación:

La Integral es:

$$\int_0^3 x^2 dx$$

Sustituido en la fórmula:

- **A** = 0.0000
- **B** = 3.0000
- **Fórmula:** $(b - a)[f(a) + f(b)]/2$
- **Sustitución:** $(3.0000 - 0.0000)[0^2 + 3^2]/2$
- **Cálculo:** $(3.0000)(9.0000)/2 = 13.500$

Resultado del método del trapecio es: 13.500

Ejercicio 2:

Código del trapecio en Python

```
def f(x):  
    return 1/x  
  
def metodo_trapecio(a, b):  
    return (b - a) * (f(a) + f(b)) / 2  
  
resultado = metodo_trapecio(1, 2)  
print(f"Resultado aproximado de la integral: {resultado:.5g}")
```

Ejecución:

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py  
Resultado aproximado de la integral: 0.75
```

Explicación:

La Integral es:

$$\int_1^2 \frac{1}{x} dx$$

Sustituido en la fórmula:

- **A** = 1.0000
- **B** = 2.0000
- **Fórmula:** $(b - a)[f(a) + f(b)]/2$
- **Sustitución:** $(2.0000 - 1.0000)[1/1 + 1/2]/2$
- **Cálculo:** $(1.0000)(1.5000)/2 = 0.75000$

Resultado del método del trapecio es: 0.75000

Ejercicio 3:

Código del trapecio en Python

```
import math

def f(x):
    return math.sqrt(x)

def metodo_trapecio(a, b):
    return (b - a) * (f(a) + f(b)) / 2

resultado = metodo_trapecio(1, 4)
print(f"Resultado aproximado de la integral: {resultado:.5g}")
```

Ejecución:

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py
Resultado aproximado de la integral: 4.5
```

Explicación:

La Integral es:

$$\int_1^4 \sqrt{x} \, dx$$

Sustituido en la fórmula:

- **A** = 1.0000
- **B** = 4.0000
- **Fórmula:** $(b - a)[f(a) + f(b)]/2$
- **Sustitución:** $(4.0000 - 1.0000)[\sqrt{1} + \sqrt{4}]/2$
- **Cálculo:** $(3.0000)(1.0000 + 2.0000)/2 = 4.5000$

Resultado del método del trapecio es: 4.5000

2. Método de Simpson 1/3

Ejercicio 1

Código de Python :

```
def f(x):  
    return x**4  
  
def simpson_un_tercio(a, b):  
    h = (b - a) / 2  
    m = (a + b) / 2  
    return (h / 3) * (f(a) + 4*f(m) + f(b))  
  
resultado = simpson_un_tercio(0, 2)  
print(f"Resultado aproximado: {resultado:.5g}")
```

Ejecución:

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py  
Resultado aproximado: 6.6667
```

Explicación:

Integral: $\int_0^2 x^4 dx$

- $a = 0.0000$, $b = 2.0000$, m (punto medio) $= 1.0000$
- $h = (2.0000 - 0.0000) / 2 = 1.0000$
- Fórmula: $\frac{h}{3} [f(a) + 4f(m) + f(b)]$
- Sustitución: $\frac{1.0000}{3} [0^4 + 4(1^4) + 2^4]$
- Cálculo: $0.33333 \times [0 + 4 + 16] = 6.6667$

Ejercicio 2

Código de Python :

```
import math

def f(x):
    return math.sin(x)

def simpson_un_tercio(a, b):
    h = (b - a) / 2
    m = (a + b) / 2
    return (h / 3) * (f(a) + 4*f(m) + f(b))

resultado = simpson_un_tercio(0, math.pi)
print(f"Resultado aproximado: {resultado:.5g}")
```

Ejecución :

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py
Resultado aproximado: 2.0944
```

Explicación:

Integral: $\int_0^{\pi} \sin(x) dx$

- **a** = 0.0000, **b** = 3.1416, **m** = 1.5708
- **h** = 1.5708
- **Sustitución:** $\frac{1.5708}{3} [\sin(0) + 4 \sin(1.5708) + \sin(3.1416)]$
- **Cálculo:** $0.52360 \times [0 + 4(1) + 0] = 2.0944$

Ejercicio 3

Código de Python :

```
def f(x):  
    return 1 / (1 + x)  
  
def simpson_un_tercio(a, b):  
    h = (b - a) / 2  
    m = (a + b) / 2  
    return (h / 3) * (f(a) + 4*f(m) + f(b))  
  
resultado = simpson_un_tercio(0, 1)  
print(f"Resultado aproximado: {resultado:.5g}")
```

Ejecución :

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py  
Resultado aproximado: 0.69444
```

Explicación:

Integral: $\int_0^1 \frac{1}{1+x} dx$

- **a** = 0.0000, **b** = 1.0000, **m** = 0.50000
- **h** = 0.50000
- **Sustitución:** $\frac{0.50000}{3} [f(0) + 4f(0.5) + f(1)]$
- **Cálculo:** $0.16667 \times [1 + 4(0.66667) + 0.5] = 0.69444$

3. Método de Simpson 3/8

Ejercicio 1

Código de Python :

```
def f(x):  
    return x**3 + 1  
  
def simpson_tres_octavos(a, b):  
    h = (b - a) / 3  
    x1 = a + h  
    x2 = a + 2*h  
    return (3 * h / 8) * (f(a) + 3*f(x1) + 3*f(x2) + f(b))  
  
resultado = simpson_tres_octavos(0, 2)  
print(f"Resultado aproximado: {resultado:.5g}")
```

Ejecución :

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py  
Resultado aproximado: 6
```

Explicación:

Integral: $\int_0^2 (x^3 + 1) dx$

- **a** = 0.0000, **b** = 2.0000
- **h** = $(2.0000 - 0.0000) / 3 = 0.66667$
- **Puntos:** $x_1 = 0.66667, x_2 = 1.3333$
- **Sustitución:** $\frac{3(0.66667)}{8} [f(0) + 3f(0.66667) + 3f(1.3333) + f(2)]$
- **Cálculo:** $0.25000 \times [1 + 3(1.2963) + 3(3.3704) + 9] = 6.0000$

Ejercicio 2

Código de Python :

```
import math

def f(x):
    return math.exp(x)

def simpson_tres_octavos(a, b):
    h = (b - a) / 3
    x1 = a + h
    x2 = a + 2*h
    return (3 * h / 8) * (f(a) + 3*f(x1) + 3*f(x2) + f(b))

resultado = simpson_tres_octavos(0, 1)
print(f"Resultado aproximado: {resultado:.5g}")
```

Ejecución :

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py
Resultado aproximado: 1.7185
```

Explicación:

Integral: $\int_0^1 e^x dx$

- **a** = 0.0000, **b** = 1.0000
- **h** = 0.33333
- **Puntos:** $x_1 = 0.33333$, $x_2 = 0.66667$
- **Sustitución:** $0.12500 \times [e^0 + 3e^{0.33333} + 3e^{0.66667} + e^1]$
- **Cálculo:** $0.12500 \times [1 + 4.1868 + 5.8431 + 2.7183] = 1.7185$

Ejercicio 3

Código de Python :

```
def f(x):  
    return 1 / (1 + x**2)  
  
def simpson_tres_octavos(a, b):  
    h = (b - a) / 3  
    x1 = a + h  
    x2 = a + 2*h  
    return (3 * h / 8) * (f(a) + 3*f(x1) + 3*f(x2) + f(b))  
  
resultado = simpson_tres_octavos(0, 1)  
print(f"Resultado aproximado: {resultado:.5g}")
```

Ejecucion :

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py  
Resultado aproximado: 0.78462
```

Explicación:

Integral: $\int_0^1 \frac{1}{1+x^2} dx$

- **a** = 0.0000, **b** = 1.0000
- **h** = 0.33333
- **Puntos:** $x_1 = 0.33333$, $x_2 = 0.66667$
- **Sustitución:** $0.12500 \times [f(0) + 3f(0.33333) + 3f(0.66667) + f(1)]$
- **Cálculo:** $0.12500 \times [1 + 3(0.9) + 3(0.69231) + 0.5] = 0.78500$

4. Diferenciación de Cinco Puntos

Ejercicio 1

Código de Python :

```
def f(x):  
    return x**4 - 3*x**2  
  
def diff_cinco_puntos(x, h):  
    # Fórmula centrada de 5 puntos  
    numerador = -f(x + 2*h) + 8*f(x + h) - 8*f(x - h) + f(x - 2*h)  
    return numerador / (12 * h)  
  
x_eval, h = 1.0, 0.1  
resultado = diff_cinco_puntos(x_eval, h)  
print(f"Resultado de la derivada: {resultado:.5g}")
```

Ejecución:

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py  
Resultado de la derivada: -2
```

Explicación:

Función: $f(x) = x^4 - 3x^2$ en $x = 1.0$

- **Puntos:** $h = 0.1$. Se usan $x \pm 0.1$ y $x \pm 0.2$.
- **Fórmula:**
$$\frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h}$$
- **Sustitución:**
$$\frac{-f(1.2) + 8f(1.1) - 8f(0.9) + f(0.8)}{1.2}$$
- **Cálculo:**
$$\frac{-(-2.2464) + 8(-2.1659) - 8(-1.7739) + (-1.5104)}{1.2} = -2.0000$$

El resultado es: -2.0000

Ejercicio 2

Código de Python :

```
import math

def f(x):
    return math.log(x) # Logaritmo natural

def diff_cinco_puntos(x, h):
    numerador = -f(x + 2*h) + 8*f(x + h) - 8*f(x - h) + f(x - 2*h)
    return numerador / (12 * h)

x_eval, h = 2.0, 0.01
resultado = diff_cinco_puntos(x_eval, h)
print(f"Resultado de la derivada: {resultado:.5g}")
```

Ejecución:

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py
Resultado de la derivada: 0.5
```

Explicación:

Función: $f(x) = \ln(x)$ en $x = 2.0$

- **Puntos:** $x = 2.0, h = 0.01$.
- **Sustitución:** $\frac{-f(2.02)+8f(2.01)-8f(1.99)+f(1.98)}{0.12}$
- **Cálculo:** Operando los valores logarítmicos con la fórmula de 5 puntos.
- **Resultado:** 0.50000

El resultado es: 0.50000

Ejercicio 3

Código de Python :

```
import math

def f(x):
    return math.exp(-x**2)

def diff_cinco_puntos(x, h):
    numerador = -f(x + 2*h) + 8*f(x + h) - 8*f(x - h) + f(x - 2*h)
    return numerador / (12 * h)

x_eval, h = 0.5, 0.1
resultado = diff_cinco_puntos(x_eval, h)
print(f"Resultado de la derivada: {resultado:.5g}")
```

Ejecución:

```
PS C:\Users\felip> & C:/Users/felip/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/felip/Downloads/trapecio.py
Resultado de la derivada: -0.7787
```

Explicación:

Función: $f(x) = e^{-x^2}$ en $x = 0.5$

- Puntos: $x = 0.5, h = 0.1$.
- Sustitución: $\frac{-f(0.7)+8f(0.6)-8f(0.4)+f(0.3)}{1.2}$
- Cálculo: $\frac{-(0.6126)+8(0.6977)-8(0.8521)+(0.9139)}{1.2} = -0.77880$

El resultado es: -0.77880

Conclusión

Tras la implementación de los diversos métodos numéricos, se puede concluir que la elección del algoritmo depende directamente de la precisión requerida y la naturaleza de la función a evaluar. Mientras que el **Método del Trapecio** ofrece una aproximación lineal sencilla, las variantes de **Simpson** proporcionan una mayor exactitud al utilizar interpolaciones parabólicas y cúbicas.

De igual forma, en la **diferenciación numérica**, se observó que la fórmula de **cinco puntos** reduce significativamente el error de truncamiento en comparación con la de tres puntos, siendo ideal para aplicaciones de ingeniería que exigen resultados rigurosos. El uso de lenguajes de programación como **Python** facilita la automatización de estos cálculos, permitiendo obtener resultados con múltiples cifras significativas de manera eficiente y confiable para la toma de decisiones técnicas.